

```

# =====
# CELL 1 – Install dependencies
# =====
!pip install -q faiss-cpu sentence-transformers transformers \
    PyPDF2 python-docx pandas tabulate torch

# =====
# CELL 2 – Imports
# =====
import os, re, textwrap
import numpy as np
import pandas as pd
import faiss

import PyPDF2
import docx

from sentence_transformers import SentenceTransformer
from transformers import pipeline
from google.colab import files

from IPython.display import display, HTML

# =====
# CELL 3 – Upload your study-material files (PDF / DOCX)
# =====
print("Upload one or more PDF / DOCX files:")
uploaded = files.upload()
print("Uploaded:", list(uploaded.keys()))

# =====
# CELL 4 – Text extraction helpers
# =====
def extract_text_pdf(path: str) -> str:
    text = []
    with open(path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        for page in reader.pages:
            t = page.extract_text()
            if t:
                text.append(t)
    return "\n".join(text)

def extract_text_docx(path: str) -> str:
    doc = docx.Document(path)
    return "\n".join(p.text for p in doc.paragraphs if p.text.strip())

def load_all_files(uploaded: dict) -> str:
    corpus = []
    for fname, data in uploaded.items():
        with open(fname, "wb") as f:
            f.write(data)
        ext = os.path.splitext(fname)[1].lower()
        if ext == ".pdf":
            text = extract_text_pdf(fname)
        elif ext in (".docx", ".doc"):
            text = extract_text_docx(fname)
        else:
            print(f" Skipping unsupported file type: {fname}")
            continue
        print(f" ✓ {fname} ({len(text):,} chars)")
        corpus.append(text)
    return "\n\n".join(corpus)

raw_text = load_all_files(uploaded)
print(f"\nTotal corpus length: {len(raw_text):,} characters")

# =====
# CELL 5 – Chunking (token-aware, with overlap)
# =====
def chunk_text(text: str, chunk_tokens: int, overlap_tokens: int = 50) -> list:
    words = text.split()
    chunks, i = [], 0
    while i < len(words):
        chunk = words[i : i + chunk_tokens]
        chunks.append(" ".join(chunk))
        i += chunk_tokens - overlap_tokens
    return [c for c in chunks if c.strip()]

```

```

return [c for c in chunks if c.strip()]

CHUNK_SIZES = {"200-token": 200, "500-token": 500}
chunks_dict = {label: chunk_text(raw_text, size) for label, size in CHUNK_SIZES.items()}

for label, cks in chunks_dict.items():
    avg = np.mean([len(c.split()) for c in cks])
    print(f"{label}: {len(cks)} chunks | avg {avg:.0f} words/chunk")

# =====
# CELL 6 – Embedding model + FAISS index builder
# =====
print("\nLoading embedding model (all-MiniLM-L6-v2) ...")
embedder = SentenceTransformer("all-MiniLM-L6-v2")

def build_faiss_index(chunks: list):
    embeddings = embedder.encode(
        chunks, show_progress_bar=True, batch_size=32, normalize_embeddings=True
    )
    dim = embeddings.shape[1]
    index = faiss.IndexFlatIP(dim)
    index.add(np.array(embeddings, dtype="float32"))
    return index, embeddings

print("\nBuilding FAISS index – 200-token chunks ...")
index_200, emb_200 = build_faiss_index(chunks_dict["200-token"])

print("\nBuilding FAISS index – 500-token chunks ...")
index_500, emb_500 = build_faiss_index(chunks_dict["500-token"])

print("\n✓ Both FAISS indexes ready.")

# =====
# CELL 7 – Retrieval helper
# =====
def retrieve(query: str, index, chunks: list, top_k: int = 5) -> list:
    q_vec = embedder.encode([query], normalize_embeddings=True)
    scores, indices = index.search(np.array(q_vec, dtype="float32"), top_k)
    return [chunks[i] for i in indices[0] if i < len(chunks)]

# =====
# CELL 8 – Transformer QA model
# =====
print("\nLoading QA model (deepset/roberta-base-squad2) ...")
qa_pipeline = pipeline(
    "question-answering",
    model="deepset/roberta-base-squad2",
    tokenizer="deepset/roberta-base-squad2",
)

MAX_CONTEXT_CHARS = 3500

def answer_question(query: str, retrieved_chunks: list) -> str:
    context = " ".join(retrieved_chunks)[:MAX_CONTEXT_CHARS]
    if not context.strip():
        return "No relevant context found."
    try:
        result = qa_pipeline(question=query, context=context)
        if result["score"] > 0.05:
            return result["answer"]
        else:
            return "Low-confidence answer – check context."
    except Exception as e:
        return f"Error: {e}"

# =====
# CELL 9 – Five questions (edit these to match your document)
# =====
QUESTIONS = [
    "What are the main topics covered in the study material?",
    "What is the definition or explanation of the core concept discussed?",
    "What examples or case studies are mentioned?",
    "What are the key takeaways or conclusions presented?",
    "What methods or techniques are described in the document?",
]

# =====
# CELL 10 – Run RAG for both chunk sizes
# =====

```

```

results = []

for q in QUESTIONS:
    ctx_200 = retrieve(q, index_200, chunks_dict["200-token"])
    ans_200 = answer_question(q, ctx_200)

    ctx_500 = retrieve(q, index_500, chunks_dict["500-token"])
    ans_500 = answer_question(q, ctx_500)

    results.append({
        "Question"           : q,
        "Answer (200-token)" : ans_200,
        "Answer (500-token)" : ans_500,
    })

print("✓ All five questions answered under both configurations.\n")

# =====
# CELL 11 – Side-by-side comparison table (HTML + plain text)
# =====
df = pd.DataFrame(results)

styled_html = f"""
<style>
    table {{ border-collapse: collapse; width: 100%; font-family: monospace; font-size: 13px; }}
    th  {{ background: #2c3e50; color: white; padding: 10px; text-align: left; }}
    td  {{ padding: 10px; vertical-align: top; border: 1px solid #bbb; }}
    tr:nth-child(even) {{ background: #f4f4f4; }}
    td:first-child {{ font-weight: bold; color: #2c3e50; width: 25%; }}
</style>
{df.to_html(index=False, escape=False, border=1)}
"""
display(HTML(styled_html))

print("\n==== PLAIN-TEXT COMPARISON =====\n")
for i, row in enumerate(results, 1):
    print(f"Q{i}: {row['Question']}")
    print(f"    [200-token] {row['Answer (200-token)']}")
    print(f"    [500-token] {row['Answer (500-token)']}")
    print(f"    -" * 80)

# =====
# CELL 12 – Configuration summary table
# =====
print("\n==== SYSTEM CONFIGURATION SUMMARY =====\n")
summary = pd.DataFrame({
    "Configuration" : ["200-token chunks", "500-token chunks"],
    "# Chunks"       : [len(chunks_dict["200-token"]), len(chunks_dict["500-token"])],
    "Overlap (tokens)" : [50, 50],
    "Embedding model" : ["all-MiniLM-L6-v2"] * 2,
    "FAISS index type" : ["IndexFlatIP (cosine sim)"] * 2,
    "QA model"         : ["deepset/roberta-base-squad2"] * 2,
    "Top-K retrieved"  : [5, 5],
})
display(HTML(summary.to_html(index=False, border=1)))

# =====
# CELL 13 – Analysis paragraph
# =====
print("""


ANALYSIS – Impact of Chunk Size on Retrieval & Answer Quality



Chunk size is one of the most influential hyperparameters in a RAG pipeline, directly governing the granularity-vs-context trade-off during retrieval. With 200-token chunks, the FAISS index holds more, tightly focused segments, which improves semantic precision – each chunk typically covers a single idea, so cosine similarity search surfaces highly targeted passages. However, this fine granularity can omit surrounding context, causing the extractive QA model to produce clipped or incomplete answers when the answer spans multiple sentences.



With 500-token chunks, each segment carries richer, self-contained context, enabling the QA model to synthesise more complete and coherent answers to explanatory or multi-step questions. The trade-off is retrieval focus: a longer chunk may contain only partial relevance to the query, diluting its embedding vector and ranking lower than a tightly scoped short chunk would. In practice, 200-token chunks excel at narrow factoid lookups, while 500-token chunks outperform on broader, reasoning-heavy questions. An optimal production system often combines both through hierarchical or adaptive chunking strategies.


""")

```

